

BudgetTracker

Firebase Setup Guide

Step-by-step: Kotlin + Firebase Realtime Database

Overview

This guide walks you through setting up Firebase for the BudgetTracker Android app. You will configure Firebase Authentication (Email/Password) and Firebase Realtime Database to store users, categories, and transactions.

Step	Task	Est. Time
1	Create a Firebase Project	2 min
2	Register your Android App	2 min
3	Enable Authentication	1 min
4	Set up Realtime Database	2 min
5	Open project in Android Studio	3 min
6	Verify Gradle dependencies	1 min
7	Set Database Security Rules	2 min
8	Run and test the app	5 min

Prerequisites

Requirement	Details
Android Studio	Hedgehog (2023.1.1) or newer recommended
Google Account	Needed to access Firebase Console
BudgetTracker project	The project zip file fully extracted
Internet connection	Required for Firebase Console and Gradle sync
JDK	Version 11 or higher (bundled with Android Studio)

Detailed Steps

Step 1: Create a Firebase Project

1. Open your browser and go to <https://console.firebase.google.com>
2. Sign in with your Google account.
3. Click the **"Add project"** button (large card on the screen).
4. Enter the project name: **BudgetTracker** then click **Continue**.
5. On the Google Analytics page, you may disable it for a student project. Click **Continue**.
6. Wait for the project to be created (~15 seconds), then click **Continue**.
7. You will land on the Firebase project dashboard.

***Note:** The project name is just a display label — it does not need to match your Android package name.*

Step 2: Register Your Android App

1. On the Firebase dashboard, click the **Android icon** (looks like the Android robot).
2. In 'Android package name' enter exactly: **com.example.budgettracker**
3. In 'App nickname' type: **BudgetTracker** (optional).
4. Leave 'Debug signing certificate' blank. Click **Register app**.
5. On the next screen click **Download google-services.json**.
6. Save this file somewhere easy to find (e.g., your Downloads folder).
7. Click **Next** through the remaining screens, then **Continue to console**.

Where to place google-services.json:

Open your BudgetTracker project folder. Place the file here:

BudgetTracker/app/google-services.json

***Important:** Replace the placeholder file already in app/. The file must be inside the app/ subfolder, NOT the root project folder.*

Step 3: Enable Email/Password Authentication

1. In the Firebase Console left sidebar, click **Build** to expand it.
2. Click **Authentication**.
3. Click **"Get started"** (first time only).
4. Click the **"Sign-in method"** tab.
5. Click on **Email/Password** in the provider list.
6. Toggle the first switch to **Enable**. Leave 'Email link' disabled.
7. Click **Save**. Email/Password will now show as Enabled.

***Note:** This is required for the Login and Register screens to work.*

Step 4: Create the Realtime Database

1. In the left sidebar click **Build** then **Realtime Database**.
2. Click **"Create Database"**.
3. Choose a database location (e.g. us-central1). Click **Next**.
4. Select **"Start in test mode"**. Click **Enable**.
5. Your database URL will look like: `https://budgettracker-xxxxx-default-rtdb.firebaseio.com`
6. Copy this URL — you may need it to verify your `google-services.json` is correct.

***Note:** Test mode allows open reads/writes for 30 days. You will secure it properly in Step 7.*

Step 5: Open Project in Android Studio

1. Open **Android Studio**.
2. Click **File > Open** and navigate to the **BudgetTracker** folder.
3. Select the folder root and click **OK**.
4. Android Studio will start **Gradle sync** automatically.
5. If prompted about JDK or SDK, use the defaults bundled with Android Studio.
6. Wait until the bottom status bar shows **"Gradle sync finished"**.

***Note:** If sync fails, verify `google-services.json` is in the `app/` folder, then try `File > Sync Project with Gradle Files`.*

Step 6: Verify Gradle Files

Your gradle files are already configured. Quickly verify these:

`app/build.gradle.kts`

- ✓ `alias(libs.plugins.google.gms.google.services)` in plugins block
- ✓ `implementation(libs.firebase.auth)` in dependencies
- ✓ `implementation(libs.firebase.database)` in dependencies

`build.gradle.kts (root)`

- ✓ `alias(libs.plugins.google.gms.google.services)` apply false in plugins block

`gradle/libs.versions.toml`

- ✓ `firebaseAuth`, `firebaseDatabase`, `firebaseStorage` versions listed
- ✓ `google-gms-google-services` plugin listed

Step 7: Set Database Security Rules

Go to **Firebase Console > Realtime Database > Rules** tab. Replace the existing rules with the following and click **Publish**:

```
{
  "rules": {
    "users": { "$uid": { ".read": "$uid === auth.uid", ".write": "$uid === auth.uid"
```

```

} },
  "transactions": { "$uid": { ".read": "$uid === auth.uid", ".write": "$uid === auth.uid" } },
  "categories": { "$uid": { ".read": "$uid === auth.uid", ".write": "$uid === auth.uid" } }
}
}

```

These rules ensure each user can only read and write their own data.

Step 8: Run and Test the App

1. In Android Studio click **Run > Run app** (or press Shift+F10).
2. Choose an emulator (API 25+) or connect a physical Android device.
3. The Splash screen appears for 3 seconds.
4. If not logged in you are taken to the **Login screen**.
5. Tap **"Sign Up"** to create a new account with email and password.
6. After registering, the app navigates to the **Dashboard**.
7. Tap the green + FAB button to add a transaction.
8. Tap '+ Category' to create a spending category first.
9. Verify data appears in Firebase Console > Realtime Database.

Firestore Realtime Database Structure

Your data is organised per authenticated user (UID):

```

budgettracker-db/
  users/
    {uid}/
      user_id, name, email, balance, created_at, main_goal
  categories/
    {uid}/
      {catId}/
        category_id, category_name, category_url, user_id
  transactions/
    {uid}/
      {transId}/
        transaction_id, transaction_name, transaction_date,
        transaction_type, transaction_amount, user_id,
        category: { category_id, category_name, ... }, receipt

```

Troubleshooting

Problem	Solution
---------	----------

Gradle sync fails	Check google-services.json is in app/ folder. Try File > Sync Project with Gradle Files.
CONFIGURATION_NOT_FOUND	The package name in google-services.json must match com.example.budgettracker exactly.
Login fails	Make sure Email/Password authentication is Enabled in Firebase Console.
Data not showing in app	Check database rules allow authenticated reads. Verify database URL.
App crashes on launch	Open Logcat in Android Studio. Look for red errors indicating missing views or null refs.
google-services.json not found	File must be named exactly google-services.json (lowercase) placed in app/ folder.
PERMISSION_DENIED	Update Realtime Database rules to allow authenticated users (Step 7).